UNIVERSITY OF OSLO

RISCY: Designing a processor using the RISC-V ISA



Yngve Hafting
Lecturer
Department of Informatics

2026-08-10

Is RISC-V DIY-able?

Background
Early decisions
RISC-V Basics
The RISCY-processor
Summary



First -

Quick poll

- Worked on - or designed a microprocessor?
  - (Raise hands)
- Write your own RTL code?
- Worked with VHDL 2019 Interfaces?

Background

Course revision, Existing vs DIY, Plan

Background-

Revising IN5200, Advanced Digital System Design

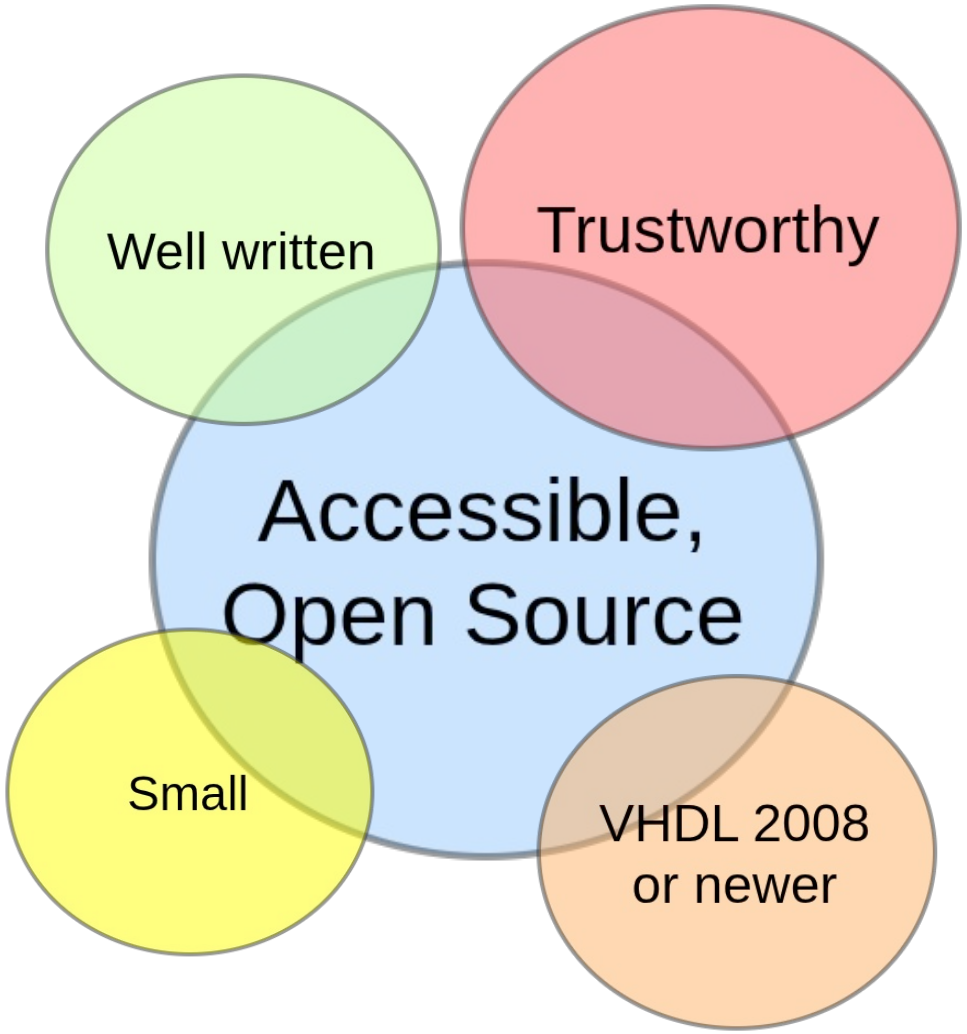
- Goal:
  - Use VHDL for design
  - Verification using cocotb
  - Assignments that can be understood
  - Use open tools and software where possible
  - Connecting to other (relevant) courses

- Building a system around a RISC-V processor seems a fun way to do lab
  - Soft core > Hard core processor
    - *If it can be read and understood*
      - *No precompiled IP!*

Existing RISC-V solutions?

Are they any good?

- Many implementations exist
  - Open?
  - Can we trust that it behaves well?
  - Well written?
    - Not generated code
  - Small enough to be read
  - VHDL
    - 2008 or newer
  - *Can we combine all these??..*

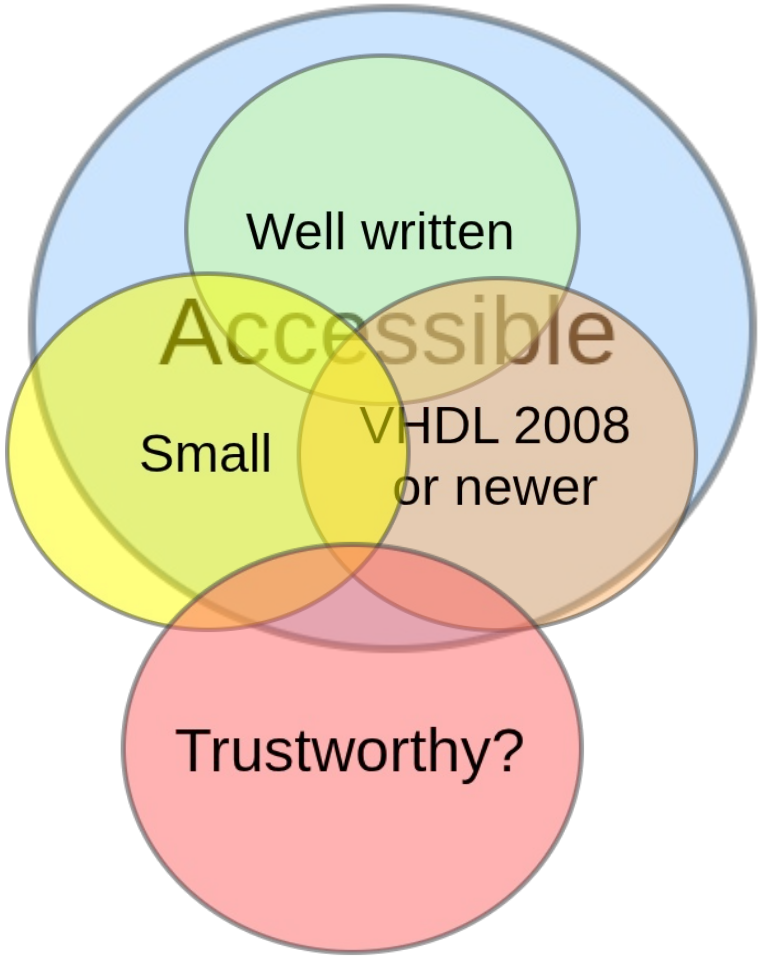


Create your own RISC-V?

Why DIY?

- Get what you want!
- Rite of passage
- Much more satisfying

- Will at least help understanding why others did what they did
- Better for teaching – *and building on*
  - May not be perfect, but
  - *Can it do the job?*



Early Decisions

Pipeline, Storage and Modularity

Early Decisions that count

Start with single cycle!

- Why not a pipeline or multi cycle?
  - Forces decisions where you might want to wait-
    - Fixed pipeline or accordion?
    - Branching / Stalls /Flushing?
      - *How to manage flow control?*
    - How to divide the project into manageable parts?
      - By opcode?
      - By state or stage?
    - What information should be kept for each stage?
  - => **affect size and readability greatly**
    - MUCH higher effort to debug!

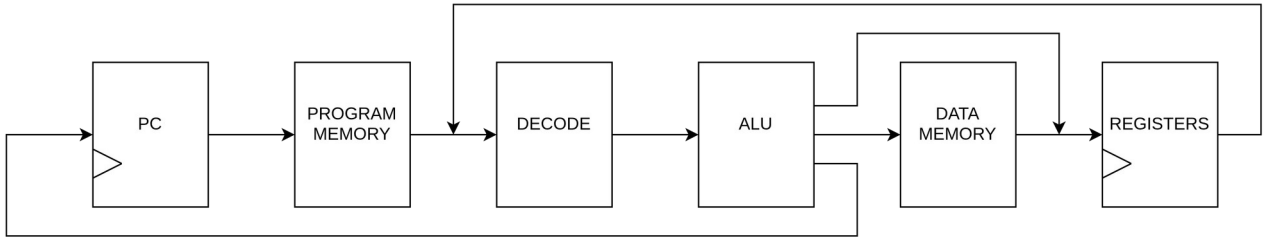
- Single cycle:
  - No flow control
  - No FSM design required
  - No storage
  - Simple testbench possible
  - Fewer modules required

Get everything up and running before diving in too deep!

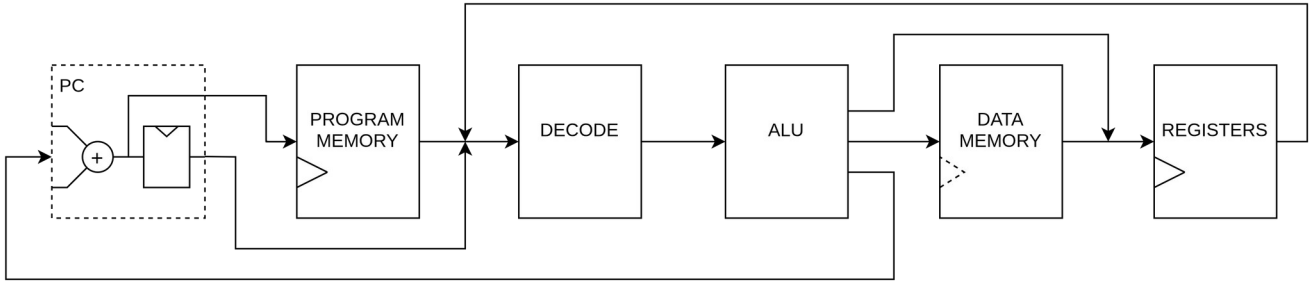
Single cycle decisions

Storage and Modularity?

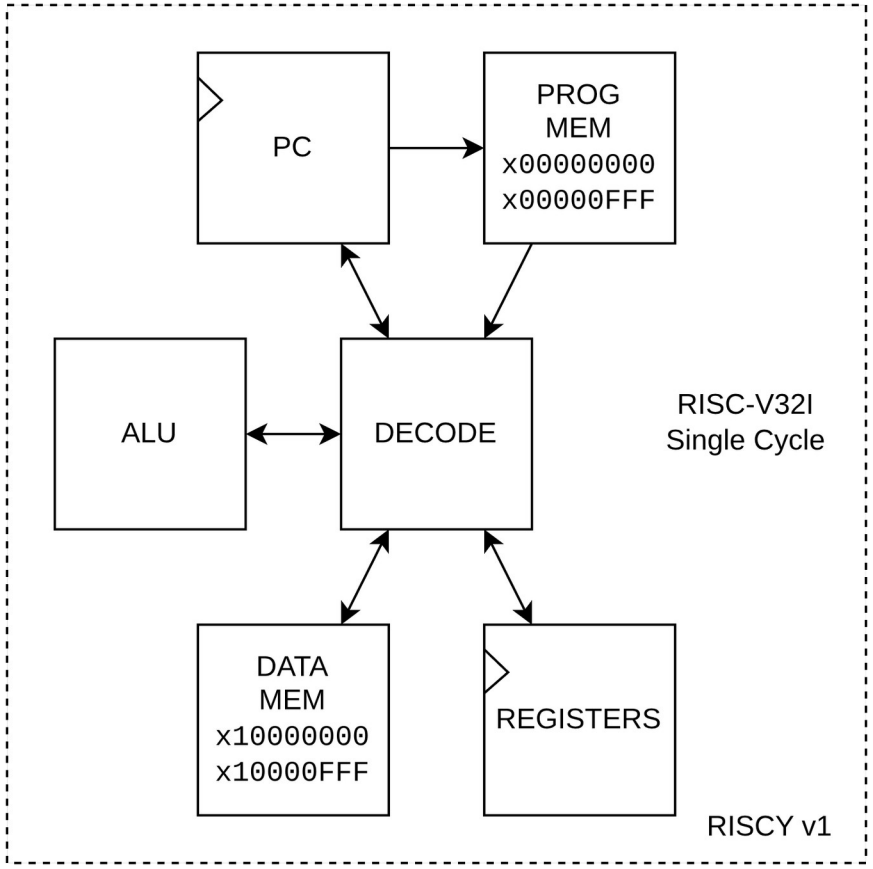
- Initial thought: everything CL, except Registers and PC



- Works with Asynchronous RAM
  - *Not FPGA-friendly =>*



- Data memory can use Dist Ram (asynch read) while Program memory can use BRAM (synch RW)
  - *Instruction pointer becomes PC-register input*



Risc-V

Basics that will get you started

What is RISC-V ISA?

Not a microarchitecture

- The ISA defines instructions and their machine code
  - May favour certain micro-architectural choices, but
  - Performing the instruction as intended = compliance
    - => Solutions may be tailor made to an application
- Opcodes for custom operations exist
  - Invites to experimentation

- Good for research, prototyping, master projects, education...

RISC-V ISA

Variants / Extensions

- Core: RISC-V32I
  - 32 bit “base Integer instruction set”
  - Defined in “The Risc-V Instruction Set Manual Volume I”
    - “Unpriveledged architecture”
  - 32 registers
  - Von Neumann architecture (single memory)
- Extensions:
  - M: Multiplication, A: Atomics, F: Floating point, D: Double Fp, Q: Quad Fp, Zicsr: Control/status reg support, Zifencei: Load/Store Fence, C: Compressed instructions (16bit), V: Vector manipulation, J: JIT compiled/interpreted

- Our focus: RV32I

RISC-V ISA

What to Read?

- ISA: 2 versions, pdf / web
  - riscv-unpriveleged.pdf : 727 pages 38 chapters
    - Only 40 pages relevant : Chapter 1-2
      - The rest is related to extensions
  - *Web : Same info = Intro + 1.1- 1.9*
- *Utilities*
  - *Risc-V reference (p1)*
    - *https://www.cs.sfu.ca/~ashriram/Courses/CS295/assets/notebooks/RISCV/RISCV_CARD.pdf*
  - Avoid stuff that is targeted programmers
    - *I read more, but did I need to?..*



UNIVERSITY OF OSLO

RISC-V Reference

RISC-V Instruction Set

Core Instruction Formats

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
funct7		rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]			rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode	B-type
imm[31:12]					rd	opcode	U-type
imm[20|10:1|11|19:12]					rd	opcode	J-type

RV32I Base Integer Instructions

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or	OR	R	0110011	0x6	0x00	rd = rs1 | rs2	
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2	msb-extends
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	zero-extends
addi	ADD Immediate	I	0010011	0x0		rd = rs1 + imm	
xori	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm	
ori	OR Immediate	I	0010011	0x6		rd = rs1 | imm	
andi	AND Immediate	I	0010011	0x7		rd = rs1 & imm	
slli	Shift Left Logical Imm	I	0010011	0x1	imm[5:11]=0x00	rd = rs1 << imm[0:4]	
srli	Shift Right Logical Imm	I	0010011	0x5	imm[5:11]=0x00	rd = rs1 >> imm[0:4]	
srai	Shift Right Arith Imm	I	0010011	0x5	imm[5:11]=0x20	rd = rs1 >> imm[0:4]	msb-extends
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0	
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0	zero-extends
lb	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]	
lh	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]	
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]	
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]	zero-extends
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]	zero-extends
sb	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]	
sh	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]	
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]	
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm	
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm	
bge	Branch ≥	B	1100011	0x5		if(rs1 >= rs2) PC += imm	
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm	zero-extends
bgeu	Branch ≥ (U)	B	1100011	0x7		if(rs1 >= rs2) PC += imm	zero-extends
jal	Jump And Link	J	1101111			rd = PC+4; PC += imm	
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm	
lui	Load Upper Imm	U	0110111			rd = imm << 12	
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)	
ecall	Environment Call	I	1110011	0x0	imm=0x0	Transfer control to OS	
ebreak	Environment Break	I	1110011	0x0	imm=0x1	Transfer control to debugger	

https://docs.riscv.org/reference/isa/v20260120/unpriv/unpriv-index.html

RISC-V ISA

What is required to design a processor?

- Specification
  - **Operations (instructions)**
    - What do each opcode do
    - What are the results
  - **Data Format**
    - What does each operation look like
  - **Storage**
    - Where does the results go
      - Registers
      - Memory
      - IO / Peripherals

- If you are going to use it:
  - Software:
    - Compilers
    - Simulators/Debuggers
    - Testbenches
  - Hardware
    - A platform that can hold an implementation (FPGA/ASIC)
      - A Hardware description…
        - VHDL, Verilog, *Chisel, System C*
      - Tools for synthesis
        - Vivado, Quartus, etc

https://docs.riscv.org/reference/isa/v20260120/unpriv/unpriv-index.html

RISC-V ISA

What are the instruction types we need?

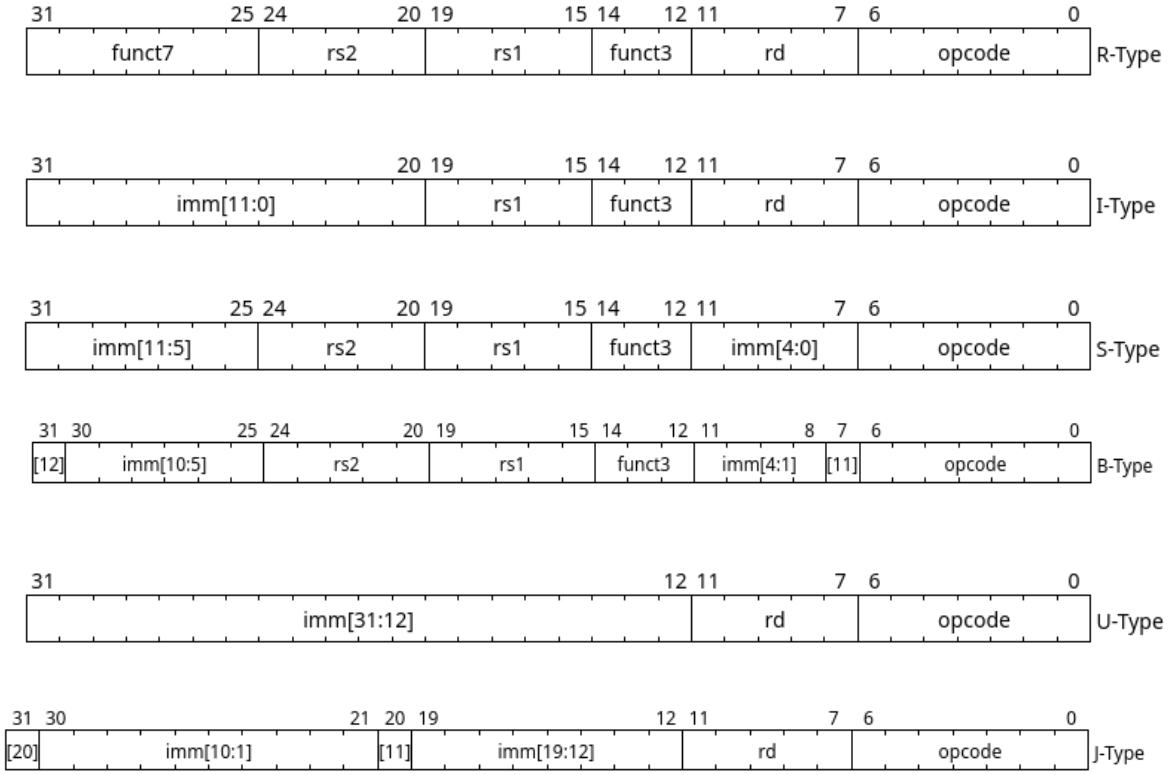
- 5 Basic instruction types:
  - **Load**
    - Move data from memory to register
  - **Store**
    - Move data from register to memory
  - **Calculate**
    - Logical operations, Add, subtract and shift
  - **Branch**
    - Select the next instruction pointer based on calculation
  - **Jump**
    - Modify the program counter (instruction pointer) to a specific value

- Each instruction type can have multiple variants and/or combinations.

RISC-V ISA

Instruction formats (RV32I unpriveleged 1.1)

- 6 instruction formats
  - 7 bit opcode +
    - 3 bit or 3+7 bit extra opcode info (funct3+7)
  - 1 or 2 source registers
  - 1 destination register
  - Immediates are crammed into rest space
    - *12 bit or 20 bit (merges to 32 bit)*



https://docs.riscv.org/reference/isa/v20260120/unpriv/unpriv-index.html

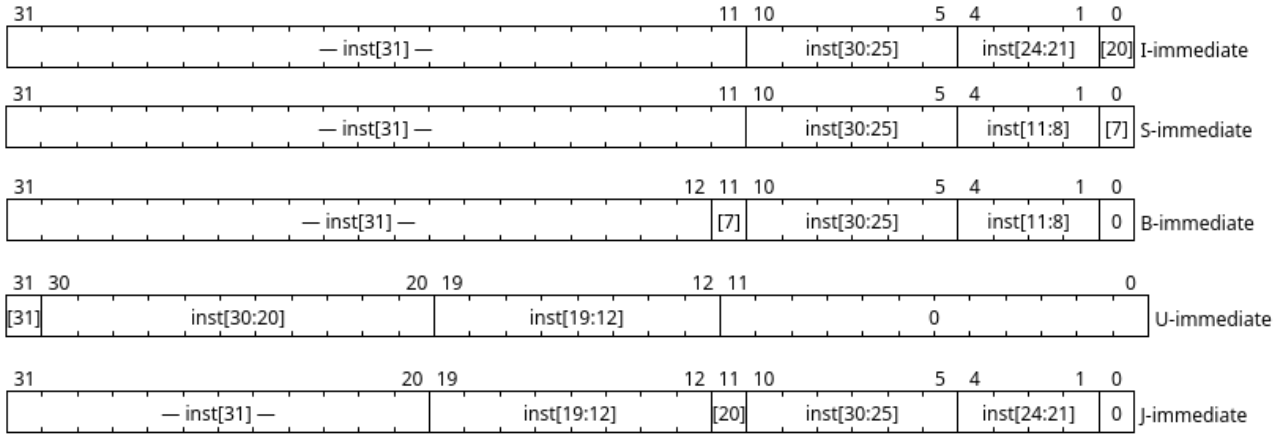
Julian Delphiki on Youtube: Why are RISC-V Immediates so weird

RISC-V ISA

Immediate formats

- Immediates are extended to 32 bit

- *This may seem like a mess*
  - *=> “reduces number of high-fanout bits”*
    - *due to sign bit extension*
  - *a low number of multiplexers is achievable-*
    - -when optimizing



RISC-V ISA

Byte Alignment or *misalignment?*

- Byte alignment means
  - 32-bit Words stored every 4th address (`address(1 downto 0) = "00"`)
  - Half words stored ever 2nd address (`address(0) = '0'`)
  - Bytes can be stored every address
- Byte alignment ensures one memory access is enough for each load/store
- RISC-V *does not* specify byte alignment
  - Words and half-word *can* be stored across word boundries
  - => Load and store may require multiple word reads
    - Either by multi-cycle or a complex memory module
- RISCY requires byte alignment

RISCY

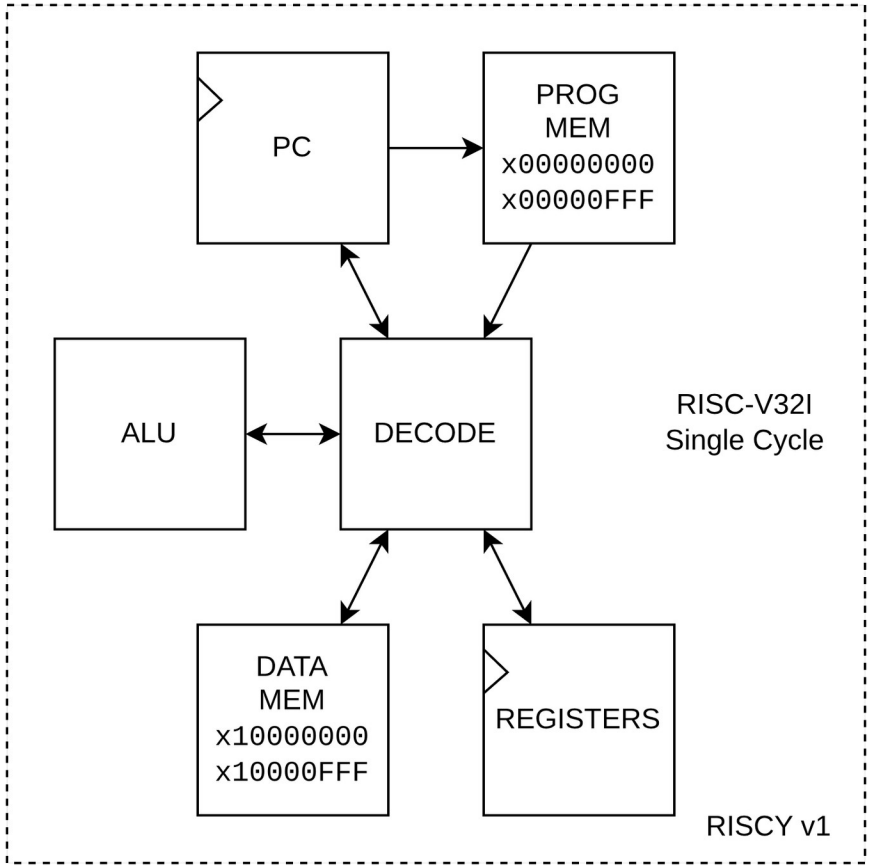
Design, verification and synthesis

RISCY, Single Cycle RISC-V Processor

Processor design

https://github.uio.no/in5200/assignments/tree/main/RISCY

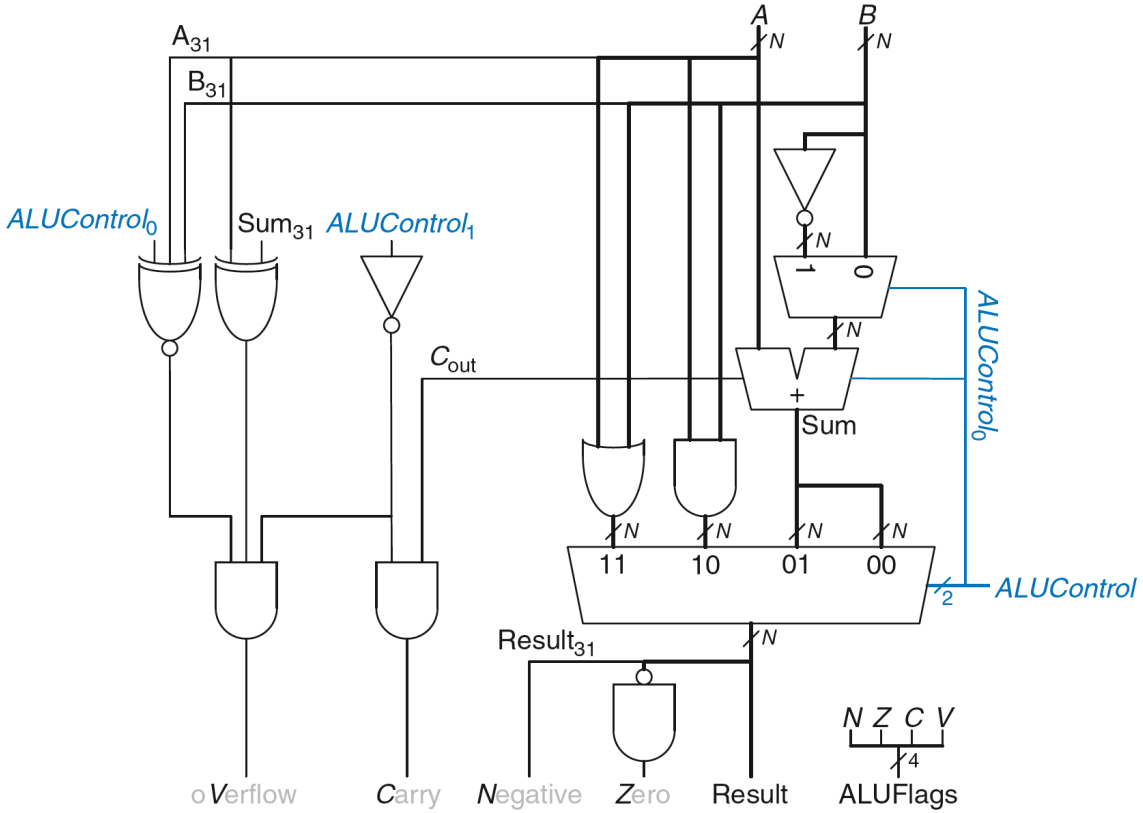
- instructions.vhdl (293 lines)
  - => Constants, types + functions for immediates
  - Coded directly from Reference Card + ISA
    - *Using types was my choice*
    - Functions for readability *not optimization*
- Interfaces.vhdl (70)
  - VHDL 2019 Interfaces
    - *Saves a LOT of trouble* see top…
- RISCY-top.vhdl (82)
  - Interfaces => Almost static when extending



RISCY Processor

Processor design

- *ALU.vhdl (48 lines)*
  - *Decided to make an ALU with flag output*
    -As IN2060/ARM, using high level code
    - *Not optimized- does add and sub separately*
- Decode.vhdl (290)
  - => opcode **`case`**
  - Mostly using the reference card directly
    - *except byte alignment*
- PC (32)
- Registers (105)
- Memory (81)



Harris and Harris: Digital Design and Computer Architecture, ARM Edition
ACM Digital Library: https://dl.acm.org/doi/10.5555/2815529

RISCY Testbench

Testbench or debugger?

- I made a simple debugger
  - Print opcode immediates and results in a single line
    - Otherwise use waveform
  - One test: *that the program counter progresses*
    - Making good tests is important going further…
  - Input data was downloaded from an online simulator
    - https://riscv-simulator-five.vercel.app/
  - I mostly checked how long RISCY stayed on target compared to the simulator
    - Checking PC progression, register values and memory

RISCY Processor

Testbench tools

- Testbench tools
  - COCOtb + NVC
    - NVC > GHDL, Questa
      - Allows deep nesting and accessing RAM data
      - Allows VHDL 2019

RISCY Processor

Synthesis

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	1921	0	0	53200	3.61
LUT as Logic	1409	0	0	53200	2.65
LUT as Memory	512	0	0	17400	2.94
LUT as Distributed RAM	512	0			
LUT as Shift Register	0	0			
Slice Registers	1023	0	0	106400	0.96
Register as Flip Flop	1023	0	0	106400	0.96
Register as Latch	0	0	0	106400	0.00
F7 Muxes	512	0	0	26600	1.92
F8 Muxes	128	0	0	13300	0.96

Site Type	Used	Fixed	Prohibited	Available	Util%
Block RAM Tile	1	0	0	140	0.71
RAMB36/FIFO*	1	0	0	140	0.71
RAMB36E1 only	1				
RAMB18	0	0	0	280	0.00

Ref Name	Used	Functional Category
FDRE	1023	Flop & Latch
LUT6	968	LUT
RAMS64E	512	Distributed Memory
MUXF7	512	MuxFx
LUT5	197	LUT
LUT2	138	LUT
MUXF8	128	MuxFx
LUT4	118	LUT
LUT3	98	LUT
OBUF	32	IO
CARRY4	26	CarryLogic
IBUF	2	IO
RAMB36E1	1	Block Memory
BUFG	1	Clock

- Vivado => Zynq7020
  - 40~50 MHz using async RAM
  - 66 MHz using DistRam only (4+4)kb
  - 62 MHz using BRAM for program memory
  - About 2-4% resources used (mostly memory + registers)
    - Using BRAM frees up much logic
- Initial multi cycle tests => 90MHz

Summary

Is RISC-V DIY-able?

- First iteration was ~2 weeks work?
  - Scattered over a longer period (Between education 2.half, spring 2026)
  - About 1000 lines RTL (including comments), 8 modules
    - + Testbench of ~400 lines
      - Mostly executing 400 clock cycles, not self testing
        - Waveform output = OK for this period
- Making decisions slows down the process…
  - Extending cost a lot more debugging – important to have a reference!
- VHDL 2019 is a requirement from here on :)
  - => Going multi cycle, adding flow-control, etc => *Virtually no change in top layer.*



UNIVERSITY
OF OSLO

Extra

Interfaces in VHDL 2019

Comparison

Top vs Top… *(2008: no flow control implemented)*

```
architecture structural-VHDL2008 of RISCY is
...

begin
...
  PROGRAM_COUNTER: entity work.program_counter
  port map(
    clk => clk, reset => reset, pc_a => pc_a, pc_b => pc_b, pc => pc );

  ALU: entity work.ALU
  port map(alu_ctrl, alu_a, alu_b, alu_result, alu_C, alu_N, alu_Z, alu_V );

  DECODE: entity work.decode
  port map(
    inst => instruction, pc => pc, pc_a => pc_a, pc_b => pc_b,
    alu_ctrl => alu_ctrl, alu_a => alu_a, alu_b => alu_b, alu_result => alu_result,
    alu_C => alu_C, alu_N => alu_N, alu_Z => alu_Z, alu_V => alu_V,
    mem_write => mem_write, mem_en => mem_en, mem_data_out => mem_din, mem_adr => mem_adr, mem_data =>
mem_dout,
    rs1 => rs1, rs2 => rs2, as1 => as1, as2 => as2, ad => ad, rd => rd );

  REGISTERS: entity work.registers
  port map(
    clk => clk,
    reset => reset,
    rs1 => rs1,
    rs2 => rs2,
    as1 => as1,
    as2 => as2,
    ad => ad,
    rd => rd );
  ...
```

```
architecture structural of RISCY is
...

begin
...
  PROGRAM_COUNTER: entity work.program_counter
  port map(
    clk => clk,
    reset => reset,
    io => pc_interface);

  ALU: entity work.ALU
  port map(clk, reset, ALU_interface);

  DECODE: entity work.decode
  port map(
    clk   => clk,
    reset => reset,
    inst  => instruction,
    pc    => pc_interface,
    alu   => ALU_interface,
    mem   => memory_interface,
    pmem  => progmem_interface,
    reg   => register_interface);

  REGISTERS: entity work.registers
  port map(
    clk => clk,
    reset => reset,
    io => register_interface);
...
```